A Voyage to the Kernel

Part 18

Segment 3.7, Day 17

We have looked at various aspects of the Linux kernel over the past issues of this magazine. We have also tried to code and load our own module. Of late, our focus has shifted to the theoretical side of kernel design and implementation. I will try to wind up the theoretical aspects over a couple of articles, and then we can devote our time to trials.

We have already seen how the system organises processes into the user and kernel space. Figure 1 summarises the overall architecture of the design we have looked at. The figure encapsulates the various concepts we've discussed— the interaction of applications with the kernel, accessing system calls, using glibc, etc. If you have missed any one of the earlier columns, you can find them at *www.linuxforu.com*.

In order to avoid some of the perplexities associated with the actual operations taking place, take a look at Figure 2, which illustrates the process in depth so that novice users can comprehend it well.

When discussing the configuration (modification) of kernel properties, I didn't mention the code that performs the action. Now we can look at some of the relevant portions that perform the related functions. If you need to review the 'global and useful constants' section, here is the associated code:

```
static ssize_t
ikconfig_read_current(struct file *file, char __user *buf,
                      size_t len, loff_t * offset)
{
    loff_t pos = *offset;
    ssize_t count;

    if (pos >= kernel_config_data_size)
        return 0;

    count = min(len, (size_t)(kernel_config_data_size - pos));
    if (copy_to_user(buf, kernel_config_data + MAGIC_SIZE + pos,
count))
        return -EFAULT;
    *offset += count;
    return count;
}
```

Now let's look at the *ikconfig_init* segment (a critical one), which does the initiation:

```
static int __init ikconfig_init(void)
{
    struct proc_dir_entry *entry;

    /* create the current config file */
    entry = create_proc_entry("config.gz", S_IFREG | S_IRUGO,
                              &proc_root);
    if (!entry)
        return -ENOMEM;

    entry->proc_fops = &ikconfig_file_ops;
    entry->size = kernel_config_data_size;

    return 0;
}
```

And finally, here is the code that performs the clean-up work:

```
static void __exit ikconfig_cleanup(void)
{
    remove_proc_entry("config.gz", &proc_root);
}

module_init(ikconfig_init);
module_exit(ikconfig_cleanup);
```

Device drivers

It is very interesting to meddle around with device drivers (DD). And it is even more exciting to write our own DD! So here in this column, I will restrict myself to some of the basic ideas concerning DD and we will come back to this when we start our trial section.

Well, as you might know, in Linux, devices are represented as files. If you have not seen this, I suggest you glance through your */dev/* directory. You might think